

We start with a simple function, `myCallbackFunction()` ❶, that just prints a message to announce that it has been called. Next, we have a `CallbackTest` class that contains the method `myMethod()` ❷, which prints its own message to announce that it's been called. We create an `oCallbackTest` object from the `CallbackTest` class ❸. We need this object so we can set up a callback to `oCallback.myMethod()`.

Then we create three `SimpleButton` objects, each using a different approach ❹. The first, `oButtonA`, has no callback. The second, `oButtonB`, sets its callback to the function `myCallbackFunction()`. The third, `oButtonC`, sets its callback to `oCallback.myMethod()`.

In the main loop, we check for the user clicking on any of the three buttons by calling the `handleEvent()` method of each button. Since `oButtonA` has no callback, we must check if the value returned is `True` ❺ and, if so, perform an action. When `oButtonB` is clicked ❻, the `myCallbackFunction()` function will be called and will print its message. When `oButtonC` is clicked ❼, the `myMethod()` method of the `oCallbackTest` object will be called and will print its message.

Some programmers prefer using a callback approach, because the target to be called is set up when you create the object. It's important to understand this technique, especially if you are using a package that requires it. However, I will use the original approach of checking for the value returned by a call to `handleEvent()` in all my demonstration code.

Summary

In this chapter, I showed how you can start with a procedural program and extract related code to build a class. We created a `Ball` class to demonstrate this, then modified the main code of our demo program from the previous chapter to call methods of the class to tell the `Ball` object *what* to do, without worrying about *how* it achieves the outcome. With all the related code in a separate class, it's easy to create a list of objects and instantiate and manage as many objects as we want to.

We then built a `SimpleButton` class and a `SimpleText` class that hide complexity inside their implementation and create highly reusable code. In